

DESIGN HANDOFF

A context-first harness for solo dev + AI agents

This document captures the design thinking behind the wireframes and gives the app team the visual system and the HTML/CSS patterns to build against. It is descriptive, not prescriptive about framework — the structure and tokens matter more than any one class name.

1 • The core bet

Traditional coding-agent UIs are a **linear stream of work**. When agents fan out and prompts are one line, the stream stops answering the only question that matters: **“what actually drove this change?”** Prompts are thin; the real drivers are **memory recalls** and **semantic/text/graph search**. So the harness is **context-first**: every view is built to trace a change back to the memory, search hits, and agent that produced it.

2 • Design principles

1. **The workstream is the unit, and it never ends.** Not sessions. A workstream is an infinite thread with a state — `active · idle · closed` — that you *pick up*, never “start over.” Its **name is inferred** from what you’re doing.
2. **The project is inferred, not chosen.** You don’t pick a project to begin. A workstream can run project-less (chat/planning); the moment work touches files in a git repo it **binds to that project** and the project name attaches to the workstream.
3. **Two nested scopes, read top-to-bottom.** **App context** lives in the header (branding + the workstream switcher box). **Workstream context** lives in the bar below it (the service nav). Everything on screen is one or the other.
4. **Consistent-but-dynamic chrome.** Header, service nav, and status line are always present; they render an *empty* state (no project, tabs locked, muted status) or a *hydrated* one. Cold start is just the empty state of the same shell.
5. **Services are first-class.** Memory, Search, and Agents are tabs, not hidden panels. A running service shows a live dot → a mini activity dropdown → a pinnable **monitor column**. Open several and the view fans into an **expanding column layout**, one pane per service. The navigator rail collapses to make room.

6. **“Search” is two different things — keep them apart.** **Literal find** (jump to a file/symbol/memory) lives in the Project page as a ⌘K box. The **Search tab** is the *audit trail of agent search operations* (semantic/text/graph), each attributed to the requesting agent — provenance, not a search field.
7. **Attribution everywhere.** Memory entries show **who requested** them; search operations show the **requesting agent**; changed lines carry an **agent tag**. Recalls are scored and marked **injected** vs **held back** so you see what actually entered context.
8. **Two file viewers, deliberately distinct.** The **project (IDE) viewer** is for browsing/editing the tree. The **activity viewer** is changed-files-first + diff with a “what drove this” panel. **Agent file artifacts link into the activity viewer** (with back-nav), because from an agent you care about the change, not the tree.
9. **Workflow guards delivery.** A per-workstream `spec → epic → issue → branch → PR → review → deploy` pipeline, newest first, side-scrolling. It explicitly flags **missing stages** and surfaces **PRs awaiting merge, unpushed commits, uncommitted files** so a fan-out can’t quietly skip review or deploy.
10. **Back-nav preserves the trail.** Any deep-link (artifact → viewer, event dot → pane) keeps a breadcrumb back to where you clicked, so exploring provenance never strands you.

3 · Visual system

Warm “engineering paper,” not a dark IDE — calm enough to live in all day, with ink-drawn borders that read as wireframe-honest.

- **Surfaces:** desk `#eceae5`, panel `#fbfaf7`, inset `#f1efe8` / `#faf8f2`; dark chrome (header/status) `#26241f`.
- **Ink & borders:** text `#2a2a2a`, hairline borders `#2a2a2a` at 1.3–1.5px (deliberately heavy, drawn).
- **Accents:** amber `#c9781f` = selection / current / “inferred”; blue `#2a78d6` = search / in-progress; green `#5aa06f` = done / active / shipped; red `#c96a6a` = gaps / blocked / dirty.
- **Type:** `system-ui` for prose and labels; `ui-monospace` for all system metadata (ids, paths, counts, status). This split is the main way the eye separates “content” from “telemetry.”

- **Motifs:** diamonds  (bound) /  (unbound) for workstreams; a 22px dotted-grid canvas behind graph/tree views; relevance bars for search hits.

4 • Suggested tokens

Lift the palette and type into variables so the empty/hydrated and light service accents stay consistent:

```
/* design tokens */:root {  
  --desk:#ecea5; --panel:#fbfaf7; --inset:#f1efe8; --chrome:#26241f;  
  --ink:#2a2a2a; --muted:#8a857a; --line:#2a2a2a;  
  --accent:#c9781f; /* selection / current / inferred */  
  --search:#2a78d6; /* search / in-progress */  
  --done:#5aa06f; /* active / shipped */  
  --gap:#c96a6a; /* blocked / missing / dirty */  
  --mono: ui-monospace, Menlo, monospace;  
  --sans: system-ui, -apple-system, sans-serif;  
}
```

5 • Shell skeleton

The whole app is one flex column: dark **header** (app context) → **service nav** (workstream context) → a **body** that splits into the collapsible navigator rail + the active pane → dark **status line**. Every screen is a variation of this.

```
<div class="app">
  <!-- APP CONTEXT: branding + workstream switcher -->
  <header class="hdr">
    <span class="logo">◆ Trusty Code</span>
    <button class="wsbox">◆ OAuth login <small>→ acme-api</small> ▼</button>
    <div class="ha">...tokens · cost · settings...</div>
  </header>

  <div class="body">
    <!-- collapsible navigator: Workstream | Project tabs -->
    <nav class="wsrail">...workstream / project list...</nav>

    <section class="actpane">
      <!-- WORKSTREAM CONTEXT: horizontal service nav -->
      <div class="wsnav">
        Workstream · Project · Agents · Memory · Search · Workflow · Files
      </div>
      <div class="actbody">...active tab / chat / monitors...</div>
    </section>
  </div>

  <footer class="statusbar">...workstream · project · mem · agents · tokens · cost...
</footer>
</div>
```

Nesting note: the `.statusbar` is a sibling of `.body` (a direct child of `.app`), never inside `.body` — otherwise it steals the rail's row width and squishes the pane. This bit us twice in the wireframe; assert it in a test.

6 • Key component CSS

Representative rules pulled from the wireframe — the parts that carry the identity.

```

/* dark chrome, both ends */
.hdr, .statusbar { background:var(--chrome); color:#e8e4da; font:11px var(--mono); }
.wsbox { background:#3a372f; border:1px solid #4a4740; border-radius:7px;
padding:4px 11px; color:#f1efe8; }
.wsbox small { color:#c9c3b6; font-weight:400; }

/* workstream-context service nav (was top tabs) */
.wsnv { display:flex; gap:2px; }
.wsnv .tab { display:flex; align-items:center; gap:6px; padding:5px 11px;
border-radius:7px; font:600 11.5px var(--sans); color:var(--muted); }
.wsnv .tab.on { background:#fff; border:1.3px solid var(--line); color:var(--ink); }
.wsnv .tab .live { width:6px; height:6px; border-radius:50%; } /* service running */
status pill, light service accents by dot color */
.statusbar .st { display:flex; gap:6px; padding:7px 13px; border-right:1px solid #3a372f;
}
.dot.g{background:var(--done)} .dot.b{background:var(--search)} .dot.a{background:var(--
accent)}

```

```

/* a service monitor (search shown; memory/agents share the frame) */
.mon { border:1.4px solid var(--line); border-radius:10px; overflow:hidden; }
.mon .head { padding:7px 11px; background:var(--inset); font:600 11px var(--mono);
display:flex; align-items:center; gap:8px; }
.hit { display:flex; gap:8px; font:10.5px var(--mono); padding:3px 6px;
border-radius:5px; background:#f7f5ef; }
.hit.top { background:#eef4ec; color:#2f6b2f; } /* best match */
.hit .bar { width:46px; height:5px; border-radius:3px; background:#e0dccf; }
.hit .bar > i { display:block; height:100%; background:var(--search); } /* relevance */
.recall.injected { border-color:var(--accent); background:#fff3e0; } /* entered context
*/

```

```

/* expanding monitor columns — one pane per open service */
.svcols { display:flex; height:100%; }
.svcol { flex:1; min-width:0; border-right:1.5px solid #d7d3ca;
display:flex; flex-direction:column; background:var(--panel); }
.svcol.chat { flex:1.25; }

/* collapsible navigator rail */
.wsrail { width:236px; transition:width .15s; }
.wsrail.collapsed { width:46px; }
.wsrail.collapsed > :not(.toggle):not(.stub) { display:none; }

```

7 • Notes on the hard parts

- **Agent-changed highlighting.** In the IDE tree, tag changed files with a colored marker (accent = modified, green = added) + edit size; in the editor gutter, tag changed lines with the agent name. The signal is “an agent touched this,” distinct from git status.
- **Artifact links.** Any file rendered next to an agent (roster, monitor, chat) is a link into the **activity viewer** with a back-nav breadcrumb — not the IDE. Deep-link to the specific hunk when possible.
- **Workflow attention.** Compute per-workstream: open PRs awaiting merge, commits ahead of remote, dirty working tree, and missing pipeline stages. Surface as a banner + per-lane chips; the lanes side-scroll independently, newest workstream first.
- **Empty states are first-class.** Cold start, project-less workstream, zero-project Project tab, and a search that returns nothing all deserve designed states — they’re the same shell, not error screens.

Pairs with: the interactive wireframe doc (turns 1–11, options linked by id) and `handoff/` (2× PNGs of screens 7a–11b). Colors are OKLCH-friendly; adjust accent saturation per platform without breaking the role mapping.